

System Design & Flutter Engine (Vol 2)

Candidate: Irshath Ahamed | Senior Flutter Developer (Portal/Mobile)

Senior Flutter Developer — Interview Master Encyclopedia (Volume 2)

Focus Areas: System Design, Hardware & Sensors, Platform Channels, Security, Dart 3 Modern Patterns, Advanced State Architecture & Live Coding

Table of Contents

- [1. Enterprise System Design in Flutter](#)
 - *System 1: Battery-Efficient Real-Time GPS Tracking & Geofencing*
 - *System 2: Multi-Tenant Role-Based Access Control (RBAC)*
 - *System 3: Secure Biometric Authentication & Hardware Keystore Flow*
 - *System 4: Real-Time Synchronization (WebSockets vs SSE vs Polling)*
- [2. Flutter Engine & Dart Internals Deep-Dive](#)
 - What `BuildContext` Actually Is Under the Hood
 - Using `BuildContext` Across Async Gaps (`mounted` checks)
 - Platform Channels: `MethodChannel` vs `EventChannel` vs Pigeon
 - Writing a Custom `RenderObject` / `RenderBox`
 - Dart 3 Core Additions: Sealed Classes, Pattern Matching & Records
- [3. Advanced State Management & Complex Edge Cases](#)
 - Inter-BLoC Communication Patterns (Without Tight Coupling)
 - State Hydration & App Restoration (`HydratedBloc`)
 - Riverpod 2.x/3.x Deep Dive (`AsyncNotifier` , `autoDispose` , `family`)
- [4. Background Services & Native OS Lifecycle](#)
 - Background Execution on Android (Foreground Services vs WorkManager)
 - Background Execution on iOS (`BGAppRefreshTask`)
 - `AppLifecycleListener` vs `WidgetsBindingObserver`
- [5. Mobile App Security & Hardening \(Enterprise Grade\)](#)
 - SSL / TLS Certificate & Public Key Hash Pinning
 - Binary Obfuscation & Symbol Stripping
 - Secure Key Management (`--dart-define` vs hardcoding)
 - OWASP Mobile Top 10 Mitigation in Flutter
- [6. Advanced Live Coding Challenge Workbook \(Set 2\)](#)
 - *Challenge 7: Mini Provider (Custom `InheritedWidget` from Scratch)*
 - *Challenge 8: Haversine Distance & Geofence Validator (Pure Dart)*
 - *Challenge 9: Exponential Backoff Retry with Jitter Stream Transformer*

- Challenge 10: Priority Async Job Queue with Concurrency Limit
- Challenge 11: Custom Flow / Tag Layout RenderObject

1. Enterprise System Design in Flutter

System 1: "Design a battery-efficient real-time GPS tracking and geofencing system in Flutter."

Interviewer Goal: Test your knowledge of mobile hardware, battery optimization, location accuracy trade-offs, and background OS limitations.

Ideal Senior Response:

"Designing location tracking requires balancing **accuracy vs battery consumption vs OS background kill policies**.

1. Accuracy & Sensor Filtering:

- We never poll GPS continuously with maximum accuracy (`LocationAccuracy.high`)—that drains a phone battery in under 3 hours.
- Instead, we use a **distance-filter and activity-recognition strategy**:
 - When stationary or inside an office geofence: Set `distanceFilter: 100` meters and reduced polling interval.
 - When movement is detected (via accelerometer or distance delta): Dynamically increase accuracy to `LocationAccuracy.balanced` with a `distanceFilter: 25` meters.

2. Geofence Calculation Strategy:

- Compute geofences using the **Haversine formula** locally on the device rather than offloading coordinate pairs to a remote server.
- If distance to the geofence perimeter is > 5km: Sleep location checks for 5–10 minutes.
- When within 200m of the boundary: Increase check frequency to detect the exact entry/exit timestamp.

3. Background OS Persistence:

- **Android:** Use a **Foreground Service** with an ongoing notification (`startForeground()`). Android 8.0+ aggressively kills background processes without an active foreground service notification.
- **iOS:** Configure `UIBackgroundModes` with `location` and set `showsBackgroundLocationIndicator = true` and `pausesLocationUpdatesAutomatically = false`.

4. Network Batching:

- Instead of firing an HTTP request on every GPS ping, buffer location coordinates into a local Hive box and flush in batches of 20 points, or immediately upon a critical geofence breach event. This prevents keeping the cellular radio in high-power state."

System 2: "How do you architect an enterprise Multi-Tenant Role-Based Access Control (RBAC) system in Flutter?"

Ideal Senior Response:

"An enterprise RBAC system in Flutter must be enforced on two distinct layers:

1. **Client-Side UI/UX Layer (Feature Gating & Navigation)**
2. **Server-Side Data Layer (Cryptographic Authorization)**

Client-Side Architecture:

- We define an immutable enum or sealed class of roles: `SUPER_ADMIN`, `ADMIN`, `MANAGER`, `EMPLOYEE`.
- Instead of checking roles directly in widgets (`if (user.role == 'ADMIN')`), we use **Permissions**:

```
enum AppPermission { canEditAttendance, canViewAuditReports, canManageUsers }
```

- We create an extension on `UserRole` returning a `Set<AppPermission>`. This makes permissions composable and flexible when business requirements evolve.
- **Route Protection via `go_router`** : In our routing configuration, we use a `redirect` guard that checks if the authenticated user's permission set includes the required permission for that route. If unauthorized, redirect to a `/forbidden` or dashboard screen.
- **Widget Guard Component:** *We create a reusable `PermissionGuard` widget:

```
class PermissionGuard extends StatelessWidget {  
  final AppPermission permission;  
  final Widget child;  
  final Widget fallback;  
  // Renders child only if currentUser.hasPermission(permission)  
}
```

Security Reality Check: Always state to the interviewer: 'Client-side RBAC is purely for user experience and navigation control. True security is enforced on the backend via JWT claim verification and PostgreSQL Row Level Security (RLS) policies so modified client binaries cannot access unauthorized database rows.' This demonstrates true senior engineering maturity."

System 3: "Design a secure Biometric Authentication and Hardware Keystore flow."

Ideal Senior Response:

"When implementing Biometric Login (FaceID / Fingerprint) in Flutter:

Step 1: Hardware Check:

- Use `local_auth` to check `canCheckBiometrics` and `isDeviceSupported()` .
- Verify that enrolled biometrics exist (`getAvailableBiometrics()`). If the user deletes all fingerprints from their phone settings, biometrics must be gracefully disabled.

Step 2: Cryptographic Storage (Never store passwords in plain text):

- We never store the user's raw password. Upon first successful email/password login, the backend issues an **OAuth Refresh Token**.
- *We store this refresh token in `flutter_secure_storage` :
 - On **Android**: Stored in the **Android Keystore**, encrypted with AES-256.
 - On **iOS**: Stored in the **iOS Keychain**, protected by `kSecAccessControlBiometryAny` .

Step 3: Biometric Challenge & Token Retrieval:

- When the user opens the app, we prompt biometric authentication:
`authenticate(localizedReason: 'Authenticate to access Fusion')` .
- If biometric authentication passes, we read the encrypted refresh token from secure storage, request a fresh short-lived JWT access token from our auth service, and hydrate the user session.
- If biometrics fail 3 times or user cancels, fall back to PIN or password authentication.
- If the device reports hardware tampering (jailbroken / rooted), lock secure storage and wipe tokens."

System 4: "Compare Real-Time Architectures: WebSockets vs Server-Sent Events (SSE) vs Long Polling in Flutter."

Technology	Directionality	Protocol	Battery / Overhead	Best Use Case in Flutter
WebSockets	Full-Duplex (Bidirectional)	TCP (<code>ws://</code> , <code>wss://</code>)	Low overhead once connection is open; persistent socket.	Real-time chat, collaborative canvas, live multiplayer, bid exchanges.
Server-Sent Events (SSE)	Unidirectional (Server to Client)	Standard HTTP (<code>text/event-stream</code>)	Lightweight, automatic reconnection built into HTTP.	Live stock prices, sports scores, build progress tickers, live audit alerts.
Short / Long Polling	Unidirectional (Client requests)	Standard HTTP REST	High battery and network overhead (repeated TCP handshakes and headers).	Fallback when WebSockets are blocked by corporate proxies or firewalls.

2. Flutter Engine & Dart Internals Deep-Dive

Q21: What is `BuildContext` under the hood?

The Technical Reality:

*" `BuildContext` is an abstract interface. In the Flutter framework, the `Element` itself implements `BuildContext` ."

When your `build(BuildContext context)` method is executed, Flutter literally passes `this` (the `Element` instance) as the `context` parameter.

That is why `BuildContext` knows its position in the tree: it IS the element node in the element tree. It is used to traverse upwards to find ancestors (`context.findAncestorWidgetOfExactType<T>()`), look up `InheritedWidget` dependencies (`Theme.of(context)`), or obtain render dimensions (`context.size`)."

Q22: Why is using `BuildContext` across async gaps dangerous, and what does `mounted` actually do?

Answer:

"When an asynchronous operation (`await fetchApi()`) is executing, the user might press the back button or navigate away. When the screen is popped, the widget's `State` object is removed from the tree and **deactivated/disposed** (`element.unmount()`).

If code executes after the `await` and accesses `context` (e.g., `Navigator.of(context).pop()` or `ScaffoldMessenger.of(context).showSnackBar()`):

- Prior to Flutter 3.7: It could cause an assertion failure: 'Looking up a deactivated widget's ancestor is unsafe' . Memory leaks occur because the closure holds a reference to a dead element.
- In modern Flutter: You must check `if (!mounted) return;` in a `StatefulWidget State`, or `if (!context.mounted) return;` for a raw `BuildContext` before using the context after any `await` ."

```
Future<void> _submitForm(BuildContext context) async {
  final result = await authRepository.login();
  // CRITICAL SENIOR CHECK:
  if (!context.mounted) return;
  Navigator.pushReplacementNamed(context, '/dashboard');
}
```

Q23: Explain Platform Channels and compare `MethodChannel` , `EventChannel` , `BasicMessageChannel` , and `Pigeon` .

- `MethodChannel` : For asynchronous **1-to-1 method invocation** between Flutter and native code (Java/Kotlin on Android, Swift/Obj-C on iOS). Examples: getting battery level, requesting native permissions.
- `EventChannel` : For **continuous streams of events** from native to Flutter. Examples: accelerometer sensors, battery percentage changes, native step counters.
- `BasicMessageChannel` : For passing raw, unstructured messages (strings, binary data) with custom message codecs.
- `Pigeon` (Senior Standard):
 - Tool by the Flutter team that eliminates hand-written string-based channel boilerplate.
 - You define an interface in a Dart spec file; Pigeon generates **type-safe** Dart, Kotlin/Java, and Swift boilerplate.

- Eliminates runtime serialization errors and type mismatches across the native boundary.
-

Q24: How do you write a Custom `RenderObject` / `RenderBox` , and what are its core pipeline methods?

When standard widgets (`Container` , `Row` , `CustomPaint`) cannot achieve a custom layout or hit-testing behavior, you extend `RenderBox` and override:

1. `performLayout()` :
 - Calculates the box's size and positions child render objects.
 - Must adhere to the Flutter Layout Rule: "**Constraints go down, Sizes go up, Parent sets position.**"
 - Calls `child.layout(childConstraints, parentUsesSize: true)` on children, then assigns `size = constraints.constrain(Size(...))` to itself.
 2. `paint(PaintingContext context, Offset offset)` :
 - Uses `context.canvas` to draw shapes or calls `context.paintChild(child, childOffset)` .
 3. `hitTest(BoxHitTestResult result, {required Offset position})` :
 - Customizes hit-testing so non-rectangular touch targets can register taps.
-

Q25: Dart 3 Core Additions: Sealed Classes, Pattern Matching & Records

Dart 3 (introduced with Flutter 3.10) fundamentally revolutionized state modeling in Flutter:

```
// 1. Sealed Classes (Exhaustive Type Hierarchy)
sealed class AttendanceState {}
class AttendanceInitial extends AttendanceState {}
class AttendanceLoading extends AttendanceState {}
class AttendanceLoaded extends AttendanceState {
    final List<AttendanceRecord> records;
    AttendanceLoaded(this.records);
}
class AttendanceError extends AttendanceState {
    final String error;
    AttendanceError(this.error);
}

// 2. Exhaustive Pattern Matching in switch expressions
Widget buildState(AttendanceState state) {
    // If you miss any subclass, Dart emits a COMPILE ERROR! No default case needed.
    return switch (state) {
        AttendanceInitial() => const Text('Ready'),
        AttendanceLoading() => const CircularProgressIndicator(),
        AttendanceLoaded(:final records) => ListView.builder(
            itemCount: records.length,
            itemBuilder: (_, i) => Text(records[i].employeeName),
        ),
        AttendanceError(:final error) => Text('Error: $error', style: const TextStyle(color: Colors.r
    });
}

// 3. Records (Anonymous, Type-safe Multiple Return Values)
(double regularHours, double otHours) calculateHours(AttendanceRecord record) {
    return (8.0, 2.5);
}

// Destructuring
final (reg, ot) = calculateHours(record);
```

3. Advanced State Management & Complex Edge Cases

Q26: How do you communicate between two independent BLoCs without tight coupling?

Anti-Pattern: Passing `BlocA` directly into `BlocB`'s constructor. This couples the two BLoCs and makes testing either BLoC in isolation painful.

Clean Enterprise Solutions:

1. Domain Layer Repository Stream (Recommended):

- Both BLoCs depend on a shared `AuthRepository` or `TimesheetRepository`.
- The repository exposes a `Stream<User>` or `Stream<List<AttendanceRecord>>`.
- `BlocA` triggers an action that updates the repository.
- `BlocB` subscribes to the repository's stream. Neither BLoC knows the other exists.

2. UI Layer Coordination via `BlocListener`:

- In the presentation widget tree:

```
BlocListener<AuthBloc, AuthState>(
  listener: (context, state) {
    if (state is Unauthenticated) {
      context.read<UserManagementCubit>().clearCache();
    }
  },
  child: child,
)
```

Q27: How does `HydratedBloc` work for state persistence?

- `HydratedBloc` automatically persists and restores BLoC states across app restarts and app termination.
- Extends standard `Bloc` / `Cubit` and requires overriding:
 - `Map<String, dynamic>? toJson(State state)`
 - `State? fromJson(Map<String, dynamic> json)`
- It uses a lightweight key-value storage engine (Hive or custom storage) in the background. When the app reopens, the state is hydrated synchronously before the first build, eliminating flash-of-loading states on cold starts.

4. Background Services & Native OS Lifecycle

Q28: How do you handle true background tasks on Android and iOS?

- **Android:**
 - **WorkManager** (`package:workmanager`): For deferred, opportunistic tasks that must execute even if the app closes (e.g., uploading cached sync logs once WiFi connects).
 - **Foreground Service**: For real-time active tasks (e.g., GPS tracking, emergency duty shift timers). Requires a sticky system tray notification to prevent the OS Low Memory Killer from terminating the process.
- **iOS:**
 - iOS does **not** allow arbitrary background processes.
 - Use `BackgroundTasks` framework (`BGAppRefreshTask` or `BGProcessingTask`).
 - The OS decides *when* to execute your background task based on device battery level, WiFi connection, and user app usage habits. Tasks are limited to a maximum execution window of 30 seconds.

Q29: What is the difference between `WidgetsBindingObserver` and `AppLifecycleListener` ?

- **`WidgetsBindingObserver` (Legacy):**
 - Requires mixing in `with WidgetsBindingObserver`, calling `WidgetsBinding.instance.addObserver(this)` in `initState()`, and removing it in `dispose()`.
 - Overrides `didChangeAppLifecycleState(AppLifecycleState state)` (`resumed`, `inactive`, `paused`, `detached`).
- **`AppLifecycleListener` (Modern - Flutter 3.13+):**

- A clean, dedicated class that doesn't require mixins.
- Exposes granular callbacks: `onResume`, `onPause`, `onHide`, `onShow`, `onDetach`, and `onExitRequested` (which allows intercepting and canceling desktop/web window close events!).

5. Mobile App Security & Hardening (Enterprise Grade)

Q30: How do you implement SSL / TLS Certificate Pinning in Flutter?

Why it matters: Prevents **Man-In-The-Middle (MITM)** attacks where an attacker installs a malicious root certificate on a user's device to intercept and inspect encrypted network payloads.

Implementation with Dio:

```
import 'dart:io';
import 'package:dio/dio.dart';
import 'package:dio/io.dart';

void setupSslPinning(Dio dio, String expectedFingerprint) {
  dio.httpClientAdapter = IOHttpClientAdapter(
    createHttpClient: () {
      final client = HttpClient(context: SecurityContext(withTrustedRoots: false));
      // Pinning via BadCertificateCallback checking SHA-256 fingerprint
      client.badCertificateCallback = (X509Certificate cert, String host, int port) {
        final certFingerprint = cert.sha256.map((b) => b.toRadixString(16).padLeft(2, '0')).join();
        return certFingerprint.toLowerCase() == expectedFingerprint.toLowerCase();
      };
      return client;
    },
  );
}
```

Q31: How do you secure sensitive API keys and secrets in Flutter?

1. **Never hardcode secrets in Git:** Never put private keys, database passwords, or JWT signing secrets in `lib/` files or `.env` files committed to version control.
2. **Use `--dart-define` or `--dart-define-from-file`:**
 - Values are injected at compile time: `flutter build apk --dart-define=API_KEY=xyz123`
 - Accessed in Dart via: `const apiKey = String.fromEnvironment('API_KEY');`
 - Because it is evaluated at compile-time as `const`, the compiler tree-shakes and embeds the string directly in the compiled binary without external readable configuration files.
3. **Backend-for-Frontend (BFF):** For high-security services (e.g., Stripe private keys or OpenAI keys), mobile apps should NEVER hold the private key. The mobile client calls your secure backend, and your backend communicates with the external provider.

Q32: How do you obfuscate a Flutter release binary?

Run build commands with `--obfuscate` and `--split-debug-info`:

```
flutter build appbundle --flavor prod --obfuscate --split-debug-info=./build/symbols
```

- **What it does:** Replaces class names, method names, field identifiers, and file paths with unreadable symbols (`a` , `b` , `c`), making reverse engineering with IDA Pro, Ghidra, or Jadx virtually impossible.
- **Symbol Files:** The `./build/symbols` directory contains the translation maps needed to de-obfuscate crash stack traces from production Crashlytics reports.

6. Advanced Live Coding Challenge Workbook (Set 2)

Challenge 7: Build a Mini Provider (`InheritedWidget`) from Scratch

Tests deep understanding of Flutter's dependency injection and element notification engine.

```

import 'package:flutter/material.dart';

/// 1. The InheritedWidget holding the state
class MiniProvider<T> extends InheritedWidget {
  final T data;

  const MiniProvider({
    super.key,
    required this.data,
    required super.child,
  });

  /// Static lookup method (equivalent to Provider.of<T>(context))
  static T of<T>(BuildContext context) {
    final provider = context.dependOnInheritedWidgetOfExactType<MiniProvider<T>>();
    assert(provider != null, 'No MiniProvider<$T> found in widget tree ancestor context');
    return provider!.data;
  }

  @override
  bool updateShouldNotify(covariant MiniProvider<T> oldWidget) {
    return oldWidget.data != data;
  }
}

/// 2. Usage Demonstration
class CounterController extends ChangeNotifier {
  int value = 0;
  void increment() {
    value++;
    notifyListeners();
  }
}

class CounterScreen extends StatefulWidget {
  const CounterScreen({super.key});
  @override
  State<CounterScreen> createState() => _CounterScreenState();
}

class _CounterScreenState extends State<CounterScreen> {
  final _controller = CounterController();

  @override
  void initState() {
    super.initState();
    _controller.addListener(() => setState(() {}));
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return MiniProvider<CounterController>(
      data: _controller,

```

```
child: Scaffold(
  body: Center(
    child: Builder(
      builder: (innerContext) {
        final controller = MiniProvider.of<CounterController>(innerContext);
        return Text('Count: ${controller.value}', style: const TextStyle(fontSize: 24));
      },
    ),
  ),
  floatingActionButton: FloatingActionButton(
    onPressed: () => _controller.increment(),
    child: const Icon(Icons.add),
  ),
),
);
}
```

Challenge 8: Haversine Distance & Geofence Validator (Pure Dart)

Tests mathematical algorithms, coordinate systems, and business validation.

```

import 'dart:math';

class Coordinate {
  final double latitude;
  final double longitude;
  const Coordinate(this.latitude, this.longitude);
}

class GeofenceValidator {
  static const double earthRadiusMeters = 6371000.0;

  /// Calculates the great-circle distance between two points in meters
  static double calculateDistanceMeters(Coordinate point1, Coordinate point2) {
    final double lat1Rad = point1.latitude * (pi / 180.0);
    final double lat2Rad = point2.latitude * (pi / 180.0);
    final double deltaLatRad = (point2.latitude - point1.latitude) * (pi / 180.0);
    final double deltaLonRad = (point2.longitude - point1.longitude) * (pi / 180.0);

    final double a = sin(deltaLatRad / 2) * sin(deltaLatRad / 2) +
      cos(lat1Rad) * cos(lat2Rad) * sin(deltaLonRad / 2) * sin(deltaLonRad / 2);

    final double c = 2 * atan2(sqrt(a), sqrt(1 - a));

    return earthRadiusMeters * c;
  }

  /// Verifies if a user coordinate is within the target work site radius
  static bool isInsideGeofence({
    required Coordinate userLocation,
    required Coordinate siteCenter,
    required double siteRadiusMeters,
    double toleranceMeters = 15.0, // GPS drift buffer
  }) {
    final distance = calculateDistanceMeters(userLocation, siteCenter);
    return distance <= (siteRadiusMeters + toleranceMeters);
  }
}

```

Challenge 9: Exponential Backoff Retry with Jitter

Tests asynchronous programming, recursion/loops, and resilience against server spikes.

```

import 'dart:async';
import 'dart:math';

Future<T> retryWithExponentialBackoff<T>({
  required Future<T> Function() operation,
  int maxRetries = 4,
  Duration baseDelay = const Duration(seconds: 1),
  Duration maxDelay = const Duration(seconds: 30),
}) async {
  int attempts = 0;
  final random = Random();

  while (true) {
    try {
      attempts++;
      return await operation();
    } catch (error) {
      if (attempts >= maxRetries) {
        rethrow;
      }

      // Calculate exponential backoff: 2^(attempts-1) * baseDelay
      final exponentialMultiplier = pow(2.0, attempts - 1).toDouble();
      final calculatedDelayMs = baseDelay.inMilliseconds * exponentialMultiplier;

      // Add Full Jitter to prevent thundering herd problem
      final jitteredDelayMs = random.nextDouble() * calculatedDelayMs;
      final finalDelay = Duration(
        milliseconds: min(jitteredDelayMs.toInt(), maxDelay.inMilliseconds),
      );

      await Future.delayed(finalDelay);
    }
  }
}

```

Challenge 10: Priority Async Job Queue with Concurrency Limit

Tests data structures, asynchronous task scheduling, and resource capping.

```

import 'dart:async';
import 'dart:collection';

class Job<T> {
  final int priority; // Higher number = higher priority
  final Future<T> Function() task;
  final Completer<T> completer;

  Job({required this.priority, required this.task, required this.completer});
}

class PriorityJobQueue {
  final int maxConcurrent;
  int _activeJobs = 0;

  // Queue sorted by priority descending
  final List<Job<dynamic>> _queue = [];

  PriorityJobQueue({this.maxConcurrent = 2});

  Future<T> addJob<T>(Future<T> Function() task, {int priority = 0}) {
    final completer = Completer<T>();
    final job = Job<T>(priority: priority, task: task, completer: completer);

    _queue.add(job);
    _queue.sort((a, b) => b.priority.compareTo(a.priority));

    _processNext();
    return completer.future;
  }

  void _processNext() {
    if (_activeJobs >= maxConcurrent || _queue.isEmpty) return;

    final job = _queue.removeAt(0);
    _activeJobs++;

    job.task().then((result) {
      job.completer.complete(result);
    }).catchError((error, stackTrace) {
      job.completer.completeError(error, stackTrace);
    }).whenComplete(() {
      _activeJobs--;
      _processNext();
    });
  }
}

```

Challenge 11: Custom Flow / Tag Layout RenderObject

Demonstrates layout math without relying on pre-built `Wrap` widgets.

```

import 'dart:math';
import 'package:flutter/rendering.dart';
import 'package:flutter/widgets.dart';

class TagFlowLayout extends MultiChildRenderObjectWidget {
  final double spacing;
  final double runSpacing;

  const TagFlowLayout({
    super.key,
    this.spacing = 8.0,
    this.runSpacing = 8.0,
    required super.children,
  });

  @override
  RenderTagFlow createRenderObject(BuildContext context) {
    return RenderTagFlow(spacing: spacing, runSpacing: runSpacing);
  }

  @override
  void updateRenderObject(BuildContext context, RenderTagFlow renderObject) {
    renderObject
      ..spacing = spacing
      ..runSpacing = runSpacing;
  }
}

class TagFlowParentData extends ContainerBoxParentData<RenderBox> {}

class RenderTagFlow extends RenderBox
  with ContainerRenderObjectMixin<RenderBox, TagFlowParentData>,
      RenderBoxContainerDefaultsMixin<RenderBox, TagFlowParentData> {
  double spacing;
  double runSpacing;

  RenderTagFlow({required this.spacing, required this.runSpacing});

  @override
  void setupParentData(RenderBox child) {
    if (child.parentData is! TagFlowParentData) {
      child.parentData = TagFlowParentData();
    }
  }

  @override
  void performLayout() {
    double currentX = 0.0;
    double currentY = 0.0;
    double currentLineMaxHeight = 0.0;
    double maxRowWidth = 0.0;

    RenderBox? child = firstChild;
    while (child != null) {
      child.layout(BoxConstraints(maxWidth: constraints.maxWidth), parentUsesSize: true);

      // Check if child overflows current line
      if (currentX + child.size.width > constraints.maxWidth && currentX > 0) {
        currentX = 0.0;
      }
    }
  }
}

```



```

        currentY += currentLineMaxHeight + runSpacing;
        currentLineMaxHeight = 0.0;
    }

    final childParentData = child.parentData as TagFlowParentData;
    childParentData.offset = Offset(currentX, currentY);

    currentX += child.size.width + spacing;
    currentLineMaxHeight = max(currentLineMaxHeight, child.size.height);
    maxRowWidth = max(maxRowWidth, currentX);

    child = childParentData.nextSibling;
}

final totalHeight = currentY + currentLineMaxHeight;
size = constraints.constrain(Size(maxRowWidth, totalHeight));
}

@override
void paint(PaintingContext context, Offset offset) {
    defaultPaint(context, offset);
}

@override
bool hitTestChildren(BoxHitTestResult result, {required Offset position}) {
    return defaultHitTestChildren(result, position: position);
}
}

```